Lab (Option A: Python): Build and Deploy a Simple Guestbook App

Skills Network

Objectives

In this lab, you will:

- Build and deploy a simple Guestbook application
- Autoscale the Guestbook application using Horizontal Pod Autoscaler
- Perform Rolling Updates and Rollbacks

Project Overview

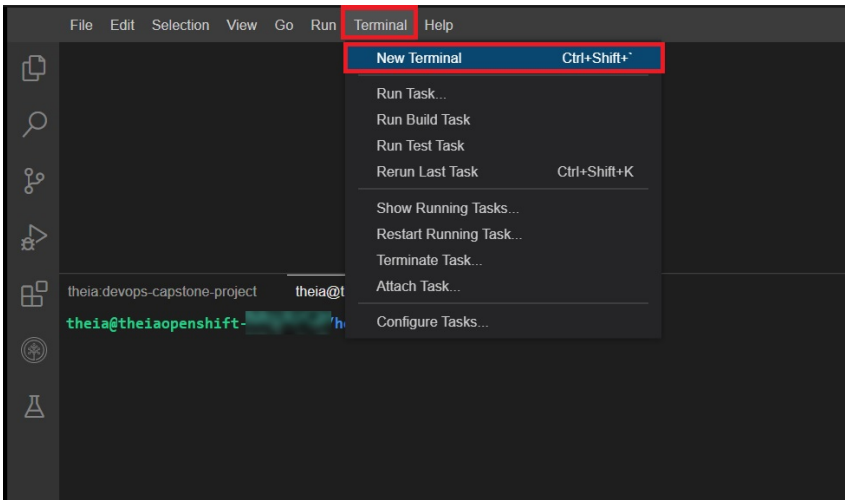
Guestbook application

Guestbook is a simple web application that we will build and deploy with Docker and Kubernetes. The application consists of a web front end, which will have a text input where you can enter any text and submit. For all of these, we will create Kubernetes Deployments and Pods. Then, we will apply Horizontal Pod Scaling to the Guestbook application and finally work on Rolling Updates and Rollbacks.

Verify the environment and command line tools

1. If a terminal is not already open, open a terminal window by using the menu in the editor: `Terminal > New Terminal`.

   Note: Please wait for some time for the terminal prompt to appear.



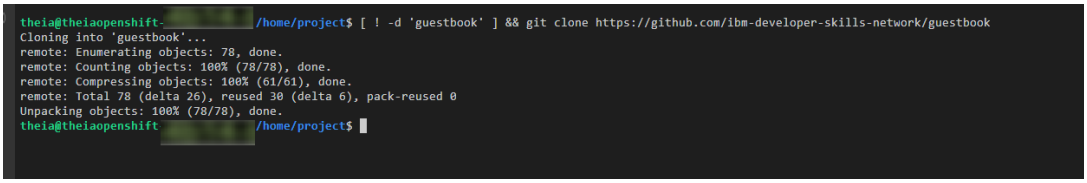
2. Change to your project folder.

   Note: If you are already on the `/home/project` folder, please skip this step.

```
cd /home/project
```

3. Clone the git repository that contains the artifacts needed for this lab.

```
[ ! -d 'guestbook' ] && git clone https://github.com/ibm-developer-skills-network/guestbook
```



4. Change to the directory for this lab.

```
cd guestbook
```

5. List the contents of this directory to see the artifacts for this lab.

```
ls
```

Creating the Guestbook app

Note:

1. Please ensure that all updates to the files are properly saved.
2. Download your project files and capture screenshots as mentioned in the respective tasks.
3. As mentioned in the *"Final Project Overview and Review Criteria"*, you can submit your project deliverables through either Option 1: AI-Graded Submission and Evaluation or Option 2: Peer-Graded Submission and Evaluation.
4. **For Option 1: AI-Graded Submission and Evaluation:**
   - Submission requires the terminal output or code snippet.
5. **For Option 2: Peer-Graded Submission and Evaluation:**
   - Submission requires screenshots for **Tasks 1 to 10**

Build the guestbook app

To begin, we will build and deploy the web front end for the guestbook app.

1. Change to the `v1/guestbook` directory.

```
cd v1/guestbook
```

2. Dockerfile incorporates a more advanced strategy called multi-stage builds, so feel free to read more about that here.

Complete the Dockerfile with the necessary Docker commands to build and push your image. The path to this file is `guestbook/v1/guestbook/Dockerfile`.

▾ Hint!
The FROM instruction initializes a new build stage and specifies the base image that subsequent instructions will build upon.
The COPY command enables us to copy files to our image.
The ADD command is used to copy files/directories into a Docker image.
The RUN instruction executes commands.
The EXPOSE instruction exposes a particular port with a specified protocol inside a Docker Container.
The CMD instruction provides a default for executing a container, or in other words, an executable that should run in your container.

Assessment:

For Option 1: AI-Graded Submission and Evaluation: Download or copy and paste the Dockerfile and save it in a text file named `Dockerfile` for the final project submission and evaluation.

For Option 2: Peer-Graded Submission and Evaluation: Take a screenshot of the completed dockerfile and save it as `Dockerfile.jpeg` or `Dockerfile.png` for the Peer Assignment.

3. Export your namespace as an environment variable so that it can be used in subsequent commands.

```
export MY_NAMESPACE=sn-labs-$USERNAME
```

```
theia@theiaopenshift-[redacted] /home/project/guestbook/v1/guestbook$ export MY_NAMESPACE=sn-labs-$USERNAME
theia@theiaopenshift-[redacted] /home/project/guestbook/v1/guestbook$
```

4. Build the guestbook app using the Docker Build command.

▾ Hint!

```
docker build . -t us.icr.io/$MY_NAMESPACE/guestbook:v1
```

```
theia@theiaopenshift-[redacted]:/home/project/guestbook/v1/guestbook$ docker build . -t us.icr.io/$MY_NAMESPACE/guestbook:v1
Sending build context to Docker daemon   98.3kB
Step 1/14 : FROM golang:1.15 as builder
1.15: Pulling from library/golang
627b765e08d1: Pull complete
c040670e5e55: Pull complete
073a180f4992: Pull complete
bf76209566d0: Pull complete
6182a456504b: Pull complete
73e3d3d88c3c: Pull complete
5946d17734ce: Pull complete
Digest: sha256:ea080cc817b02a946461d42c02891bf750e3916c52f7ea8187bccde8f312b59f
Status: Downloaded newer image for golang:1.15
 ---> 40349a2425ef
Step 2/14 : RUN go get github.com/codegangsta/negroni
 ---> Running in f0ae405f6c93
Removing intermediate container f0ae405f6c93
 ---> ff3c71a6a901
Step 3/14 : RUN go get github.com/gorilla/mux github.com/xyproto/simpleredis
 ---> Running in 5414403396b8
Removing intermediate container 5414403396b8
 ---> 414074738399
Step 4/14 : COPY main.go .
 ---> 7a25b8c69819
Step 5/14 : RUN go build main.go
 ---> Running in 8f6951d3434b
Removing intermediate container 8f6951d3434b
 ---> aa17b6f49c23
Step 6/14 : FROM ubuntu:18.04
18.04: Pulling from library/ubuntu
08a6abff8943: Pull complete
Digest: sha256:982d72c16416b09ffd2f71aa381f761422085eda1379dc66b668653607969e38
Status: Downloaded newer image for ubuntu:18.04
 ---> f5cbed4244ba
Step 7/14 : COPY --from=builder /go//main /app/guestbook
 ---> dab533567691
```

5. Push the image to IBM Cloud Container Registry.

▾ Hint!

```
docker push us.icr.io/$MY_NAMESPACE/guestbook:v1
```

```
theia@theiaopenshift-[redacted]:/home/project/guestbook/v1/guestbook$ docker push us.icr.io/$MY_NAMESPACE/guestbook:v1
The push refers to repository [us.icr.io/sn-labs-[redacted]/guestbook]
0791a8c653f4: Pushed
fd382c0f22b7: Pushed
aeb42d7f3c54: Pushed
54deed58a6a4: Pushed
5dddb4b5996f: Pushed
95c443da13bf: Pushed
v1: digest: sha256:459dc3814b3b0d59aee57f684db436c69abfa52a492fbed84e75c6c42188fe40 size: 1570
theia@theiaopenshift-[redacted]:/home/project/guestbook/v1/guestbook$
```

Note: If you have tried this lab earlier, there might be a possibility that the previous session is still persistent. In such a case, you will see a **'Layer already Exists'** message instead of the **'Pushed'** message in the above output. We recommend you to proceed with the next steps of the lab.

6. Verify that the image was pushed successfully.

```
ibmcloud cr images
```

```
theia@theiaopenshift-lavanyar: /home/project/guestbook/v1/guestbook ×

theia@theiaopenshift-[redacted]/home/project/guestbook/v1/guestbook$ ibmcloud cr images
Listing images...

Repository                                    Tag      Digest         Namespace          Created          Size     Security status
us.icr.io/sn-labs-[redacted]/guestbook        v1       61e06d36213c   sn-labs-[redacted] 21 minutes ago   32 MB    -
us.icr.io/sn-labsassets/instructions-splitter latest   2af122cfe4ee   sn-labsassets      2 years ago      21 MB    -
us.icr.io/sn-labsassets/pgadmin-theia         latest   0adf67ad81a3   sn-labsassets      2 years ago      101 MB   -
us.icr.io/sn-labsassets/phpmyadmin            latest   b66c30786353   sn-labsassets      2 years ago      163 MB   -
us.icr.io/sn-labsassets/tts-standalone        latest   56ce85280714   sn-labsassets      1 week ago       3.7 GB   -

OK
theia@theiaopenshift-[redacted]/home/project/guestbook/v1/guestbook$
```

Assessment:

For Option 1: AI-Graded Submission and Evaluation: Copy and paste the terminal output and save it in a text file named `crimages` for the final project submission and evaluation.

For Option 2: Peer-Graded Submission and Evaluation: Take a screenshot of the output of Step 6 and save it as `crimages.jpeg` or `crimages.png` for the Peer Assignment.

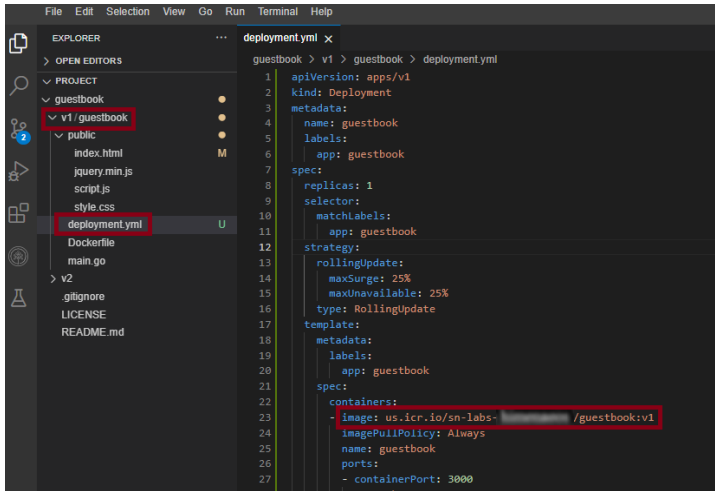
7. Open the `deployment.yml` file in the `v1/guestbook` directory and view the code for the deployment of the application:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: guestbook
  labels:
    app: guestbook
spec:
  replicas: 1
  selector:
    matchLabels:
      app: guestbook
  strategy:
    rollingUpdate:
      maxSurge: 25%
      maxUnavailable: 25%
    type: RollingUpdate
  template:
    metadata:
      labels:
        app: guestbook
    spec:
      containers:
      - image: us.icr.io/<your sn labs namespace>/guestbook:v1
        imagePullPolicy: Always
        name: guestbook
```

```
ports:
- containerPort: 3000
  name: http
resources:
  limits:
    cpu: 50m
  requests:
    cpu: 20m
```

Note: Replace `<your sn labs namespace>` with your SN labs namespace. To check your SN labs namespace, please run the command `ibmcloud cr namespaces`

- It should look as below:



8. Apply the deployment using:

```
kubectl apply -f deployment.yml
```

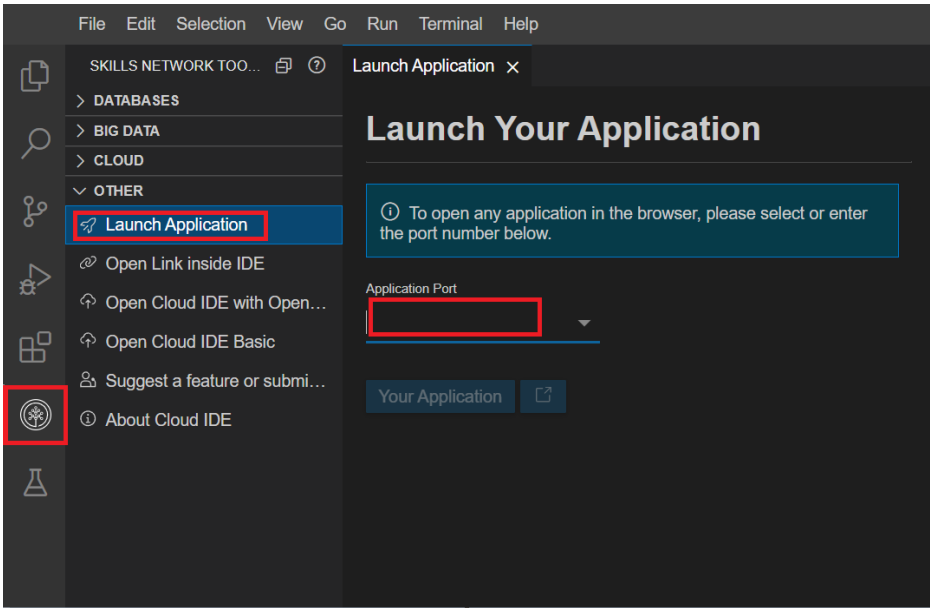
```
theia@theiaopenshift :/home/project/guestbook/v1/guestbook$ kubectl apply -f deployment.yml
deployment.apps/guestbook configured
```

9. Open a New Terminal and enter the below command to view your application:

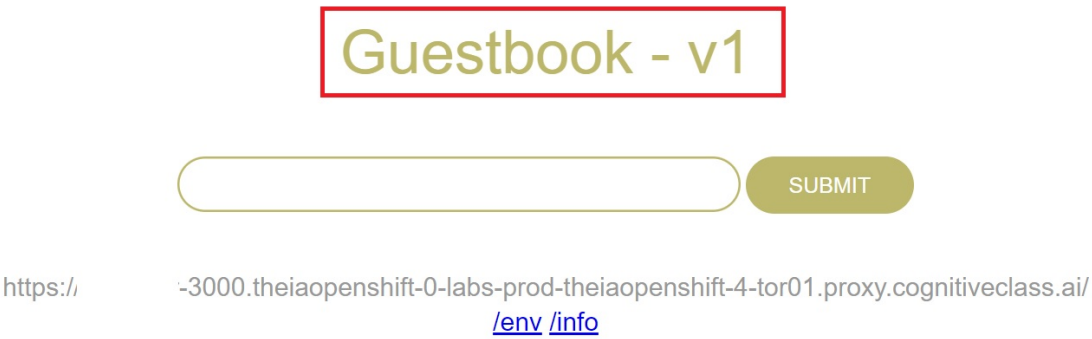
```
kubectl port-forward deployment.apps/guestbook 3000:3000
```

```
theia@theiaopenshift :/home/project/guestbook/v1/guestbook$ kubectl port-forward deployment.apps/guestbook 3000:3000
Forwarding from 127.0.0.1:3000 -> 3000
Forwarding from [::1]:3000 -> 3000
Handling connection for 3000
Handling connection for 3000
Handling connection for 3000
Handling connection for 3000
Handling connection for 3000
Handling connection for 3000
Handling connection for 3000
Handling connection for 3000
Handling connection for 3000
Handling connection for 3000
Handling connection for 3000
```

10. Launch your application on port 3000. Click on the Skills Network button on the right, it will open the **"Skills Network Toolbox."** Then, click the **Other** then **Launch Application.** From there, you should be able to enter the port and launch.



11. Now you should be able to see your running application. Please copy the app URL which will be given as below:



Assessment:

For Option 1: AI-Graded Submission and Evaluation: Copy and paste the code of the file index.html and save it in a text file named `app` for the final project submission and evaluation. You will be required to provide the code from this file, showing the default title and header.

For Option 2: Peer-Graded Submission and Evaluation: Take a screenshot of your deployed application and save it as `app.jpeg` or `app.png` for the Peer Assignment.

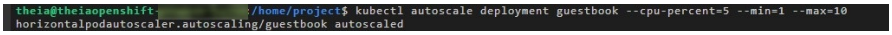
12. Try out the guestbook by putting in a few entries. You should see them appear above the input box after you hit **Submit**.

Autoscale the Guestbook application using Horizontal Pod Autoscaler

1. Autoscale the Guestbook deployment using `kubectl autoscale deployment`.

▾ Hint!

```
kubectl autoscale deployment guestbook --cpu-percent=5 --min=1 --max=10
```



2. You can check the current status of the newly-made HorizontalPodAutoscaler, by running:

```
kubectl get hpa guestbook
```

The current replicas is 0 as there is no load on the server.

Assessment:

For Option 1: AI-Graded Submission and Evaluation: Copy and paste the terminal output and save it in a text file named `hpa` for the final project submission and evaluation.

For Option 2: Peer-Graded Submission and Evaluation: Take a screenshot of your Horizontal Pod Autoscaler and save it as `hpa.jpeg` or `hpa.png` for the Peer Assignment.

3. Open another new terminal and enter the below command to generate load on the app to observe the autoscaling. (Please ensure your port-forward command is running. In case you have stopped your application, please run the port-forward command to re-run the application at port 3000.)

```
kubectl run -i --tty load-generator --rm --image=busybox:1.36.0 --restart=Never -- /bin/sh -c "while sleep 0.01; do wget -q -O- <your app URL>; done"
```

- Please replace your app URL in the `<your app URL>` part of the above command.

  Note: Use the same copied URL which you obtained in step 11 of the previous task.

The command will be as below:

```
theia@theiaopenshift:/home/project$ kubectl run -i --tty load-generator-7 --rm --image=busybox:1.28 --restart=Never -- /bin/sh -c "while sleep 0.01; do wget -q -O- https://300
0.theiaopenshift-2-labs-prod-theiaopenshift-4-tor01.proxy.cognitiveclass.ai/; done"
```

Note: In case you get a `Load generator already exists` error, please suffix a number after `load-generator`, for example, `load-generator-1`, `load-generator-2`.

- You will keep getting an output similar as below which will indicate the increasing load on the app:

```
    </div>

    <div id="guestbook-entries">
      <link href="https://afeld.github.io/emoji-css/emoji.css" rel="stylesheet">
      <p>Waiting for database connection... <i class='em em-boat'></i></p>

    </div>

    <div>
      <form id="guestbook-form">
        <input autocomplete="off" id="guestbook-entry-content" type="text">
        <a href="#" id="guestbook-submit">Submit</a>
      </form>
    </div>

    <div>
      <p><h2 id="guestbook-host-address"></h2></p>
      <p><a href="env">/env</a>
      <a href="info">/info</a></p>
    </div>
    <script src="jquery.min.js"></script>
    <script src="script.js"></script>
  </body>
</html>
```

Note: Continue further commands in the 1st terminal.

4. Run the below command to observe the replicas increase in accordance with the autoscaling:

```
kubectl get hpa guestbook --watch
```

```
theia@theiaopenshift:/home/project$ kubectl get hpa guestbook --watch
NAME        REFERENCE              TARGETS         MINPODS   MAXPODS   REPLICAS   AGE
guestbook   Deployment/guestbook   <unknown>/5%   1         10        0          51s
```

5. Run the above command again after 5-10 minutes and you will see an increase in the number of replicas which shows that your application has been autoscaled.

```
^Ctheia@theiaopenshift:/home/project$ kubectl get hpa guestbook --watch
NAME        REFERENCE              TARGETS         MINPODS   MAXPODS   REPLICAS   AGE
guestbook   Deployment/guestbook   <unknown>/5%   1         10        0          2m54s
guestbook   Deployment/guestbook   <unknown>/5%   1         10        1          3m31s
```

Assessment:

For Option 1: AI-Graded Submission and Evaluation: Copy and paste the terminal output of your Autoscaler details and save it in a text file named `hpa2` for the final project submission and evaluation.

For Option 2: Peer-Graded Submission and Evaluation: Take a screenshot of your Autoscaler details and save it as `hpa2.jpeg` or `hpa2.png` for the Peer Assignment.

6. Run the below command to observe the details of the horizontal pod autoscaler:

```
kubectl get hpa guestbook
```

```
^Ctheia@theiaopenshift:/home/project$ kubectl get hpa guestbook
NAME        REFERENCE              TARGETS         MINPODS   MAXPODS   REPLICAS   AGE
guestbook   Deployment/guestbook   <unknown>/5%   1         10        1          8m2s
```

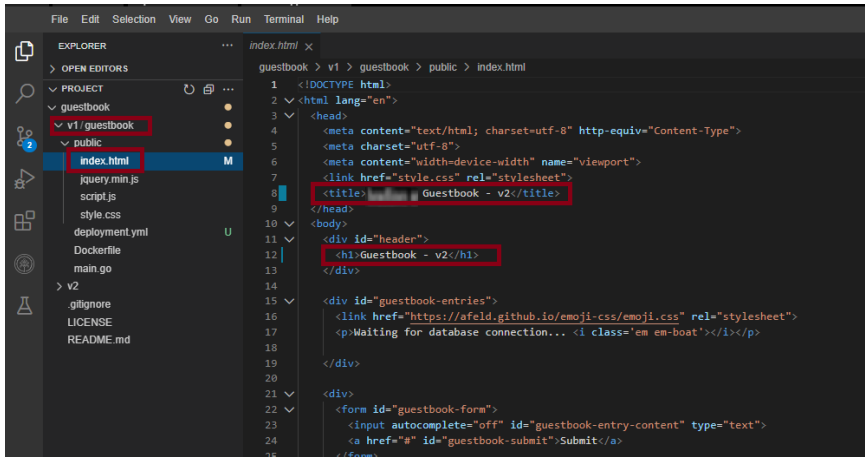
- Please close the other terminals where load generator and port-forward commands are running.

Perform Rolling Updates and Rollbacks on the Guestbook application

Note: Please run all the commands in the 1st terminal unless mentioned to use a new terminal.

1. Please update both the title and header in `index.html` to **Guestbook – v2**.

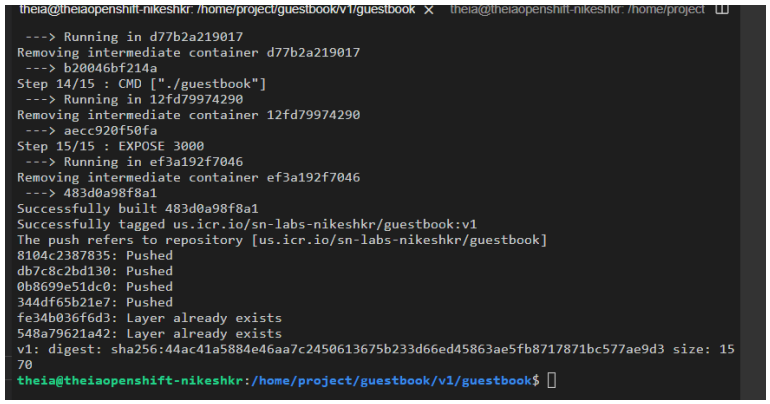
▾ Hint!



2. Run the below command to build and push your updated app image:

▾ Hint!

```
docker build . -t us.icr.io/$MY_NAMESPACE/guestbook:v1 && docker push us.icr.io/$MY_NAMESPACE/guestbook:v1
```



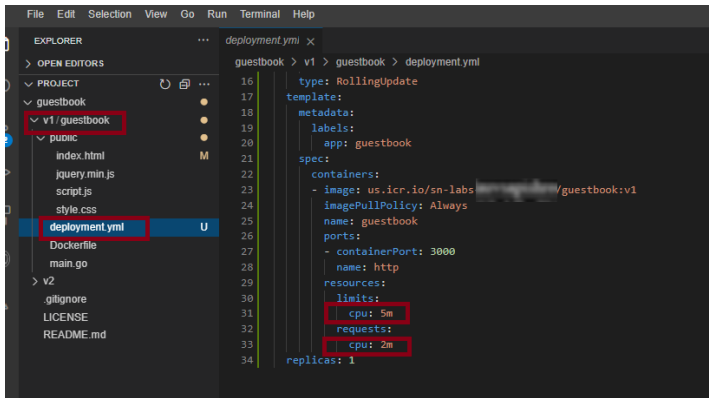
Assessment:

For Option 1: AI-Graded Submission and Evaluation: Copy and paste the terminal output and save it in a text file named `upguestbook` for the final project submission and evaluation.Your saved output must include the final digest line shown after a successful push.

For Option 2: Peer-Graded Submission and Evaluation: Take a screenshot of your updated image and save it as `upguestbook.jpeg` or `upguestbook.png` for the Peer Assignment.

3. Update the values of the CPU in the `deployment.yml` to **cpu: 5m** and **cpu: 2m** as below:

▾ Hint!



4. Apply the changes to the `deployment.yml` file.

▾ Hint!

```
kubectl apply -f deployment.yml
```

```
theia@theiaopenshift:/home/project/guestbook/v1/guestbook$ kubectl apply -f deployment.yml
deployment.apps/guestbook configured
```

Assessment:

For Option 1: AI-Graded Submission and Evaluation: Copy and paste the terminal output of Step 4 and save it in a text file named `deployment` for the final project submission and evaluation.

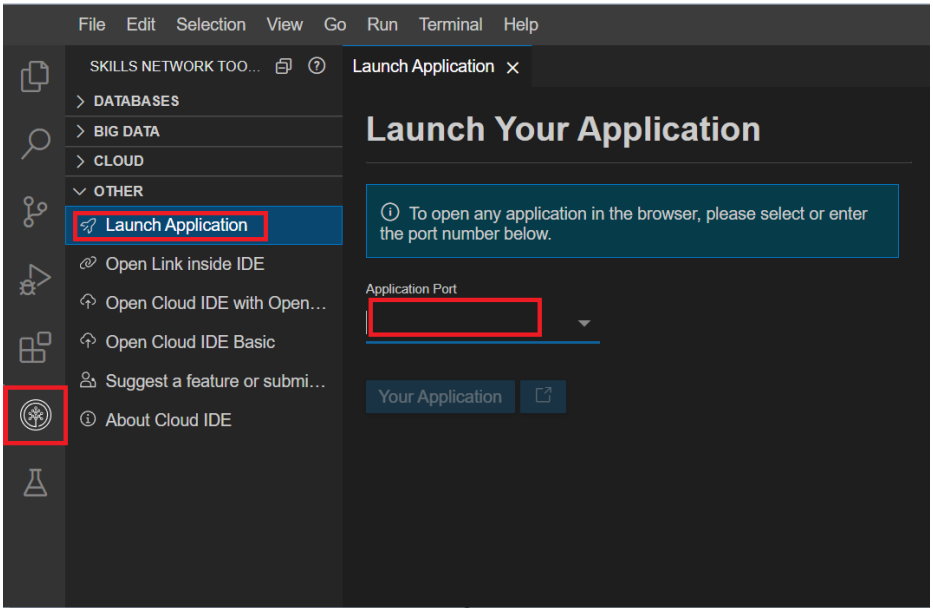
For Option 2: Peer-Graded Submission and Evaluation: Take a screenshot of the details of the output of Step 4 and save it as `deployment.jpeg` or `deployment.png` for the Peer Assignment.

5. Run the port-forward command again to start the app:

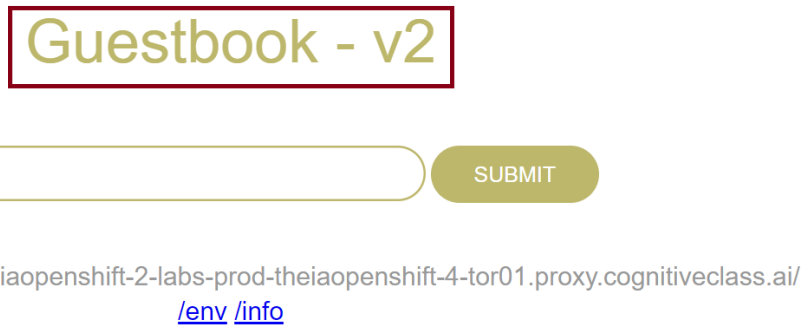
```
kubectl port-forward deployment.apps/guestbook 3000:3000
```

```
theia@theiaopenshift:/home/project/guestbook/v1/guestbook$ kubectl port-forward deployment.apps/guestbook 3000:3000
Forwarding from 127.0.0.1:3000 -> 3000
Forwarding from [::1]:3000 -> 3000
Handling connection for 3000
Handling connection for 3000
Handling connection for 3000
Handling connection for 3000
Handling connection for 3000
Handling connection for 3000
Handling connection for 3000
Handling connection for 3000
Handling connection for 3000
Handling connection for 3000
Handling connection for 3000
```

6. Launch your application on port 3000. Click on the Skills Network button on the right, it will open the **"Skills Network Toolbox."** Then, click the **Other** then **Launch Application**. From there you should be able to enter the port and launch.



7. You will notice the updated app content as below:



Assessment:

For Option 1: AI-Graded Submission and Evaluation: Copy and paste the code of your updated index.html and save it in a text file named `up-app` for the final project submission and evaluation. You'll be asked to paste the updated code from index.html showing the title and header as "Guestbook – v2".

For Option 2: Peer-Graded Submission and Evaluation: Take a screenshot of your updated application and save it as `up-app.jpeg` or `up-app.png` for the Peer Assignment.

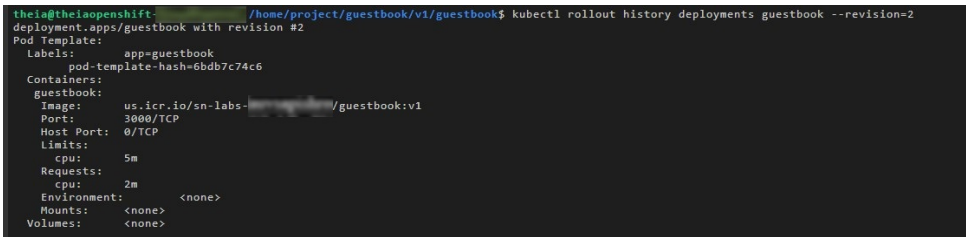
Note: Please stop the application before running the next steps.

8. Run the below command to see the history of deployment rollouts:

```
kubectl rollout history deployment/guestbook
```

9. Run the below command to see the details of Revision of the deployment rollout:

```
kubectl rollout history deployments guestbook --revision=2
```



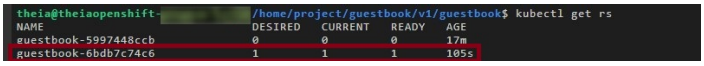
Assessment:

For Option 1: AI-Graded Submission and Evaluation: Copy and paste the terminal output of the details of the correct Revision and save it in a text file named `rev` for the final project submission and evaluation.

For Option 2: Peer-Graded Submission and Evaluation: Take a screenshot of the details of the correct Revision and save it as `rev.jpeg` or `rev.png` for the Peer Assignment.

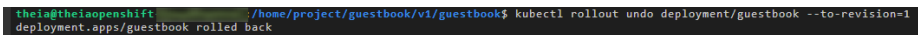
10. Run the below command to get the replica sets and observe the deployment which is being used now:

```
kubectl get rs
```



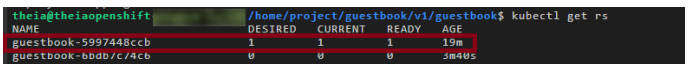
11. Run the below command to undo the deployment and set it to Revision 1:

```
kubectl rollout undo deployment/guestbook --to-revision=1
```



12. Run the below command to get the replica sets after the Rollout has been undone. The deployment being used would have changed as below:

```
kubectl get rs
```



Assessment:

For Option 1: AI-Graded Submission and Evaluation: Copy and paste the terminal output and save it in a text file named `rs` for the final project submission and evaluation.

For Option 2: Peer-Graded Submission and Evaluation: Take a screenshot of the output of Step 12 and save it as `rs.jpeg` or `rs.png` for the Peer Assignment.

Note: The rollout may take a few minutes to stabilize. Please wait until the replica set matches the state shown in the screenshot above before copying the output for future reference.

Congratulations! You have completed the final project for this course. Do not log out of the lab environment (you can close the browser though) or delete any of the artifacts created during the lab, as these will be needed for the next lab,`Optional: Deploy Guestbook App from the OpenShift Internal Registry`.

Author(s)

Manvi Gupta